

Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica



Carrera: Ingeniería en Computación

Profesor: Óscar Manuel Ruíz Hurtado

Curso: Sistemas Distribuidos

S08 - Bitácora de MexiBanco: Que la transferencia no deje dinero en el aire

Alumno: Rodríguez Zamora Joshua

Fecha de entrega: 16/10/2026

Semestre: 2027-1

Grupo: 02

Contenido

1. **El problema:** 3
2. **Tu saga:** 7
3. **El código clave:** 9
4. **La compensación funcionando:** 12
5. **Cuando la compensación también falla:** 15
6. **El circuit breaker:** 18
7. **Lo que la saga no te da:** 20

1. El problema:

Mostrar los saldos antes y después de la Parte 0, y en una frase por qué pasó.

Primero que todo, preparamos el entorno en POSTMAN para poder ejecutar las pruebas adecuadamente, por lo que declaramos:

- Saldo Inicial para CLABE: 123456789012345678:



imagen 1. Saldo inicial de cuenta origen.

- Saldo inicial para la CLABE 1010101010101011:

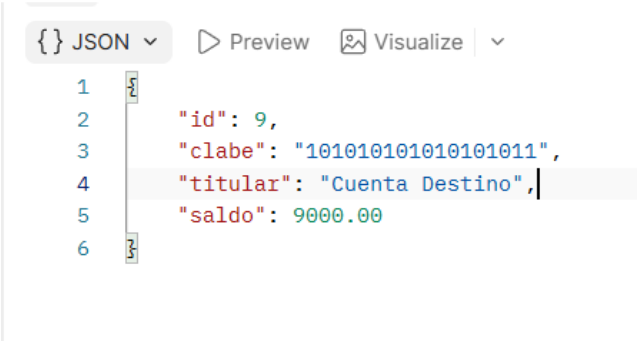


imagen 2. Saldo inicial de cuenta destino.

En base a estas cuentas es que se desarrollará esta práctica, usaremos las opciones y funcionalidades de mexi-banco para poder transferir dinero y comprobar el estado de cada transferencia para añadir nuevas funcionalidades sobre la marcha, pero primero habremos de definir el problema que tenemos con mexi-banco al inicio. El cual se genera con la pregunta. ¿Qué sucede cuando se cae el servicio de cuentas a media transferencia?

Este error lo podemos generar de la siguiente manera:

En Postman se genera el POST que hará la transferencia utilizando la lógica de mexi-banco, permitiendo de esta manera que se tengan las consideraciones básicas y/o necesarias para generar un traspaso de dinero entre cuentas.

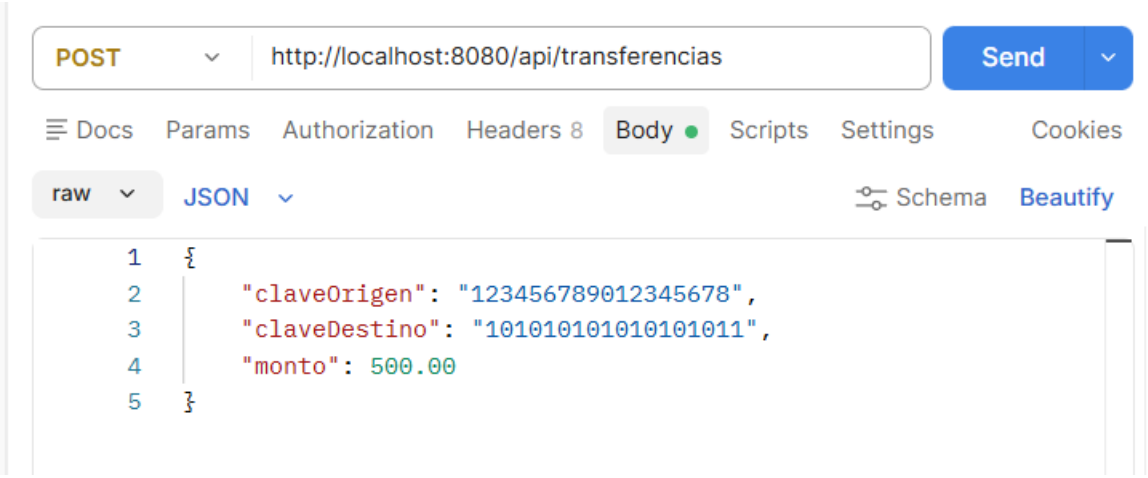


imagen 3. POST a transferencias.

Además, cambiaremos el tiempo de pausa que ocurre entre el cargo y el abono a 20 segundos, otorgándonos el tiempo adecuado para detener el servicio de cuentas en Docker de la siguiente forma:

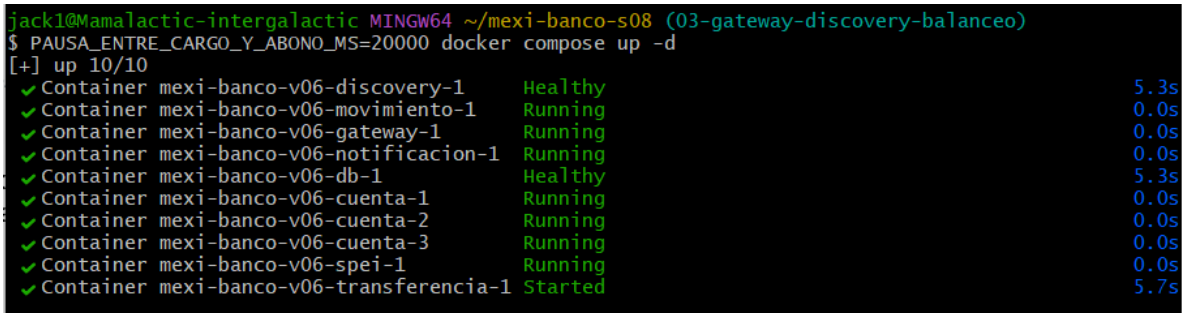


imagen 4. Asignando pausa y reiniciando servicios.

Al asignar el tiempo diseñado, podemos realizar la prueba, esto lo realizaremos corriendo el POST de la transferencia e inmediatamente detener el servicio de cuenta, evitando así que se genere la transferencia por no poder encontrar la cuenta destino.

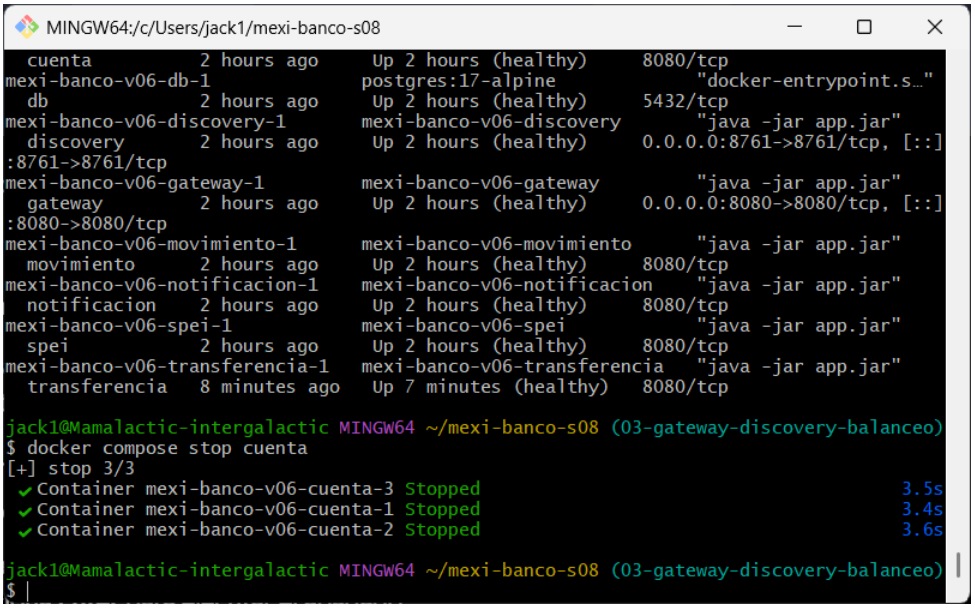


imagen 5. Servicio cuenta detenido.

Al hacer esto, POSTMAN nos arrojará el siguiente error, que implica que la operación fallo y no se completó:

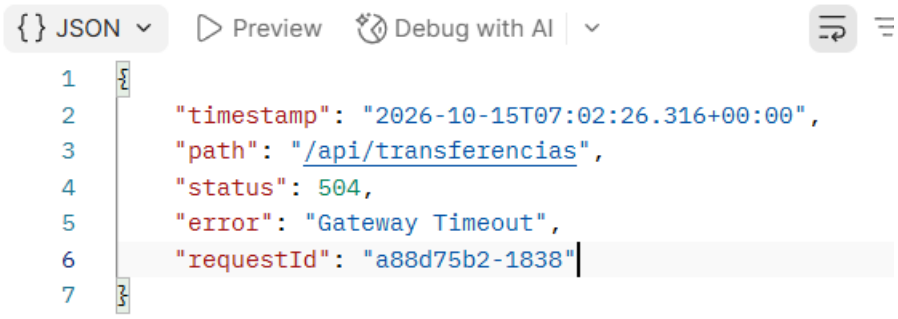


imagen 6. Error 504, gateway timeout.

Ahora, al levantar el servicio “cuenta” nuevamente y revisar los saldos de cada cuenta podemos comprobar que, sí se generó el cargo de la cuenta origen, sin embargo, no hubo un abono a la cuenta destino, lo que significa que efectivamente y como se plantea en el ejercicio, el dinero quedó volando:

- Cuenta origen con 500 pesos menos:

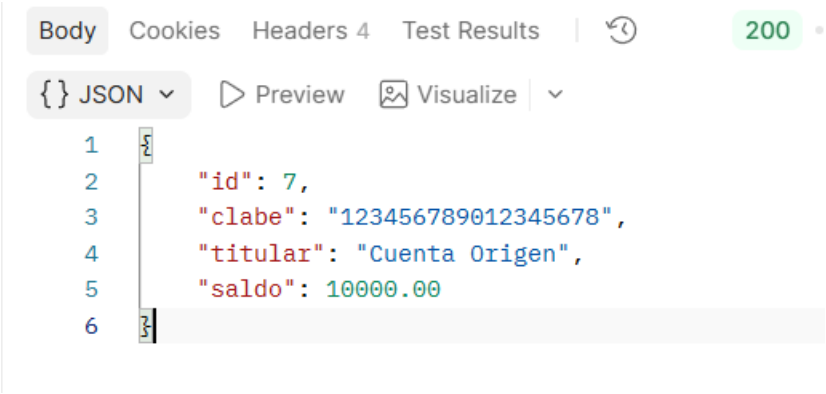


imagen 7. Cuenta origen.

- Cuenta destino sin recibir ese dinero (saldo igual al pasado):



imagen 8. Cuenta destino.

2. Tu saga:

los pasos y la compensación de cada uno, con un **diagrama de estados** de la transferencia (a mano y fotografiado también vale).

1. **Registro inicial:** Válida las cuentas de origen y destino y crea el registro de la transferencia en la base de datos. Cambia el estado a INICIADA.

2. **Cargo al origen:** El orquestador hace una petición HTTP al microservicio cuenta para restar dinero de la cuenta origen cambiando así el estado a CARGADA.

3. **Abono al destino:** El orquestador hace una petición a Cuenta para sumar el dinero a la cuenta destino. Si este abono es exitoso, el estado pasa a ABONADA, se registra el movimiento y se cambia finalmente a COMPLETADA.

   De lo contrario, si el abono falla, ya sea porque una cuenta no existe, el servicio cayó o el sistema tuvo un fallo interno, se activa la compensación, es aquí donde el orquestador ejecuta la petición reversa a cuenta, para devolver el dinero a la cuenta origen, registrando el movimiento y pasando al estado COMPENSADA.

   Por otro lado, si al intentar compensar la transacción ocurre un error, como que tras 3 reintentos aún no se pueda devolver el dinero, el sistema ya no intentará devolver el dinero y marcará la operación con el estado COMPENSACIÓN_PENDIENTE, en donde se requiere intervención humana.

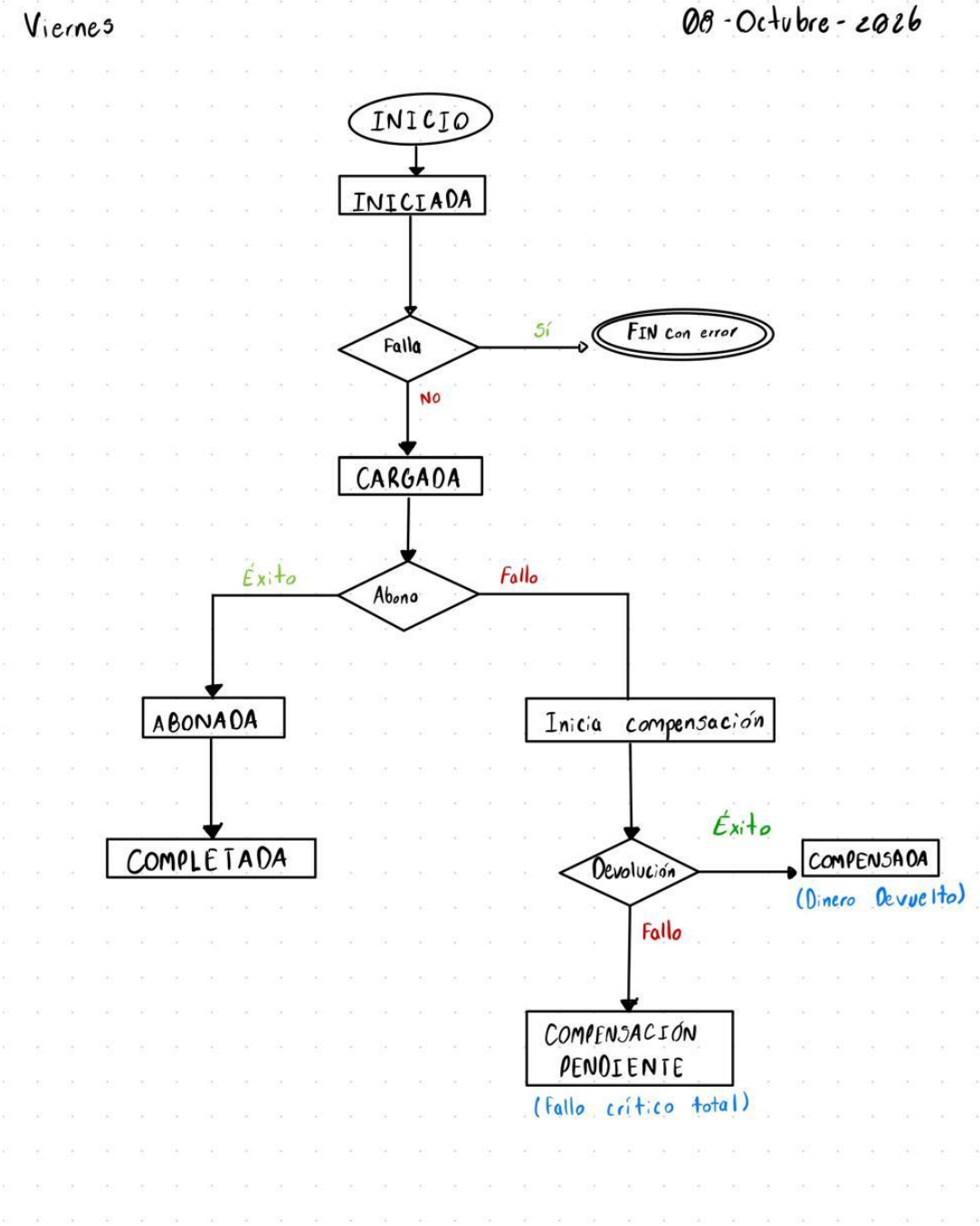


imagen 9. Diagrama de estados.

3. El código clave:

los fragmentos que cambiaste, explicados con tus palabras.

Primero se generó un nuevo código, que define al circuit breaker, en este se incorporan condiciones para añadir 3 estados fundamentales del circuito; el cerrado, el abierto y el semi abierto. Estos se definen de la siguiente forma:

- **Circuito cerrado:** Todo funciona como se esperaba. Las peticiones llegan adecuadamente hacia el servicio externo.

- **Circuito abierto:** Se activa cuando se superó el límite de errores permitidos (3). El circuito corta la conexión y rechaza nuevas peticiones sin intentar conectarse, dándole una ventana de tiempo al servicio caído para recuperarse.

- **Circuito Semi abierto:** Ha pasado el tiempo definido en Tiempo_espera_ms, por lo que el circuito permite pasar una sola petición de prueba.

En este código se utiliza synchronized para que en caso de que lleguen muchísimas peticiones al mismo tiempo, el contador de fallos no se corrompa.

En el caso de tener un circuito abierto, el código revisa la hora actual, buscando cuanto del tiempo de espera ha pasado, es decir, revisar si la falla dejó el tiempo esperado de tolerancia para que el servicio pueda recuperarse, y en ese momento cambia el estado a semi abierto.

Se usan try y catch para manejar errores, tanto en la red como en la ejecución, sumando 1 al contador fallas. Si se alcanzan los 3 errores consecutivos, o si la única petición de prueba (circuito semi abierto) vuelve a fallar, el circuito entra en estado ABIERTO y guarda la hora en la que se entró a dicho estado para volver a evaluar el estado contemplando el estado.

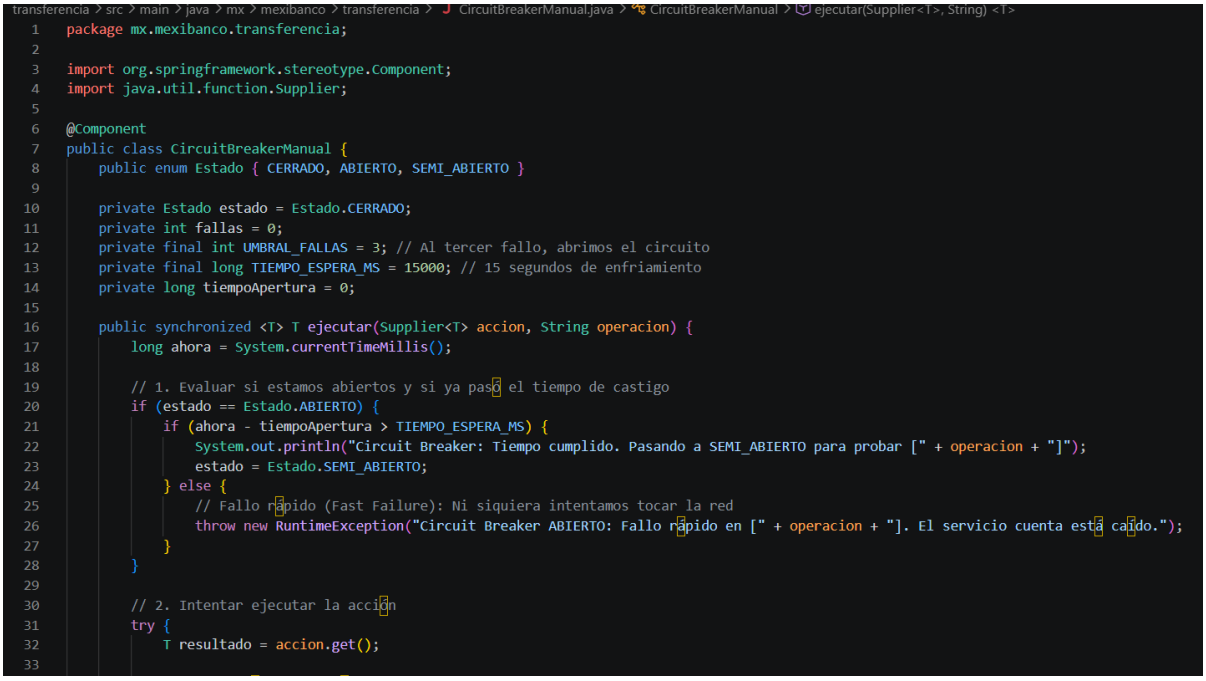


imagen 10. Código CB.

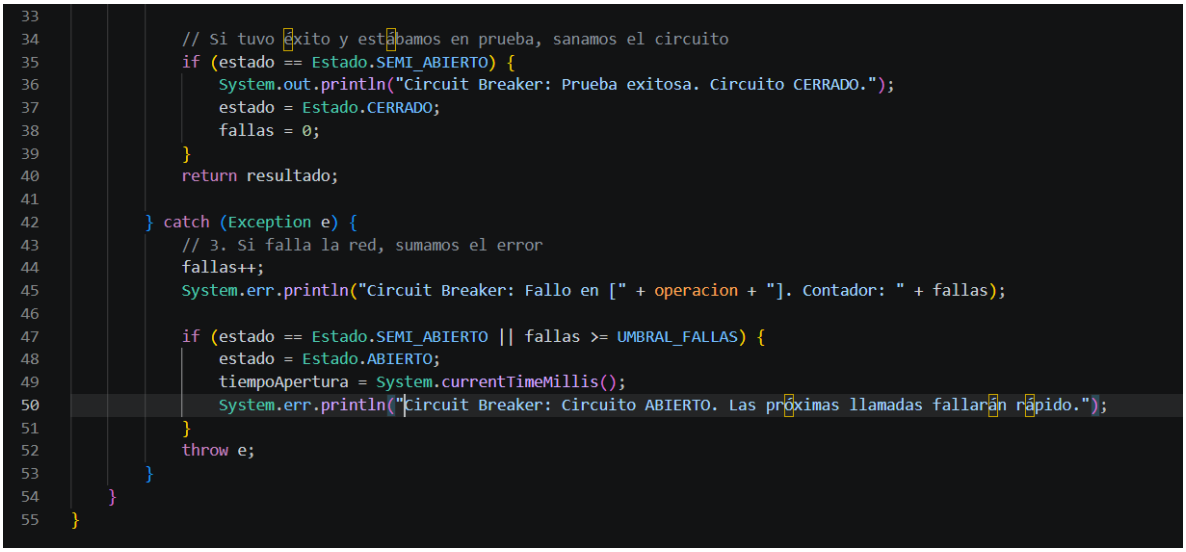
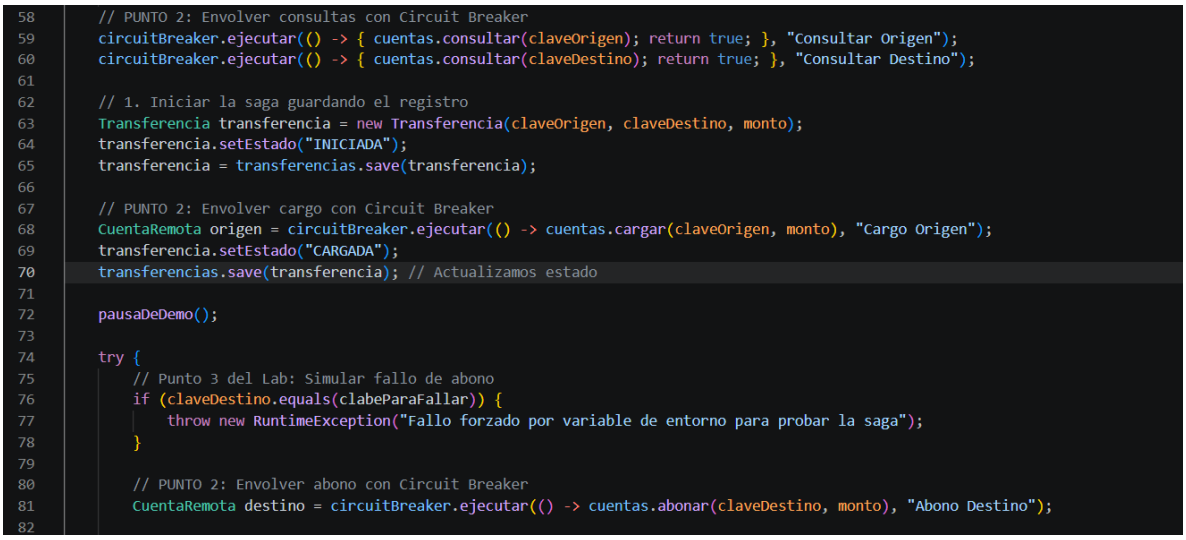


imagen 11. Código CB 2.

Ahora, el código modificado en TransferenciaService.java es el siguiente:



```
58      // PUNTO 2: Envolver consultas con Circuit Breaker
59      circuitBreaker.ejecutar(() -> { cuentas.consultar(claveOrigen); return true; }, "Consultar Origen");
60      circuitBreaker.ejecutar(() -> { cuentas.consultar(claveDestino); return true; }, "Consultar Destino");
61
62      // 1. Iniciar la saga guardando el registro
63      Transferencia transferencia = new Transferencia(claveOrigen, claveDestino, monto);
64      transferencia.setEstado("INICIADA");
65      transferencia = transferencias.save(transferencia);
66
67      // PUNTO 2: Envolver cargo con Circuit Breaker
68      CuentaRemota origen = circuitBreaker.ejecutar(() -> cuentas.cargar(claveOrigen, monto), "Cargo Origen");
69      transferencia.setEstado("CARGADA");
70      transferencias.save(transferencia); // Actualizamos estado
71
72      pausaDeDemo();
73
74      try {
75          // Punto 3 del Lab: Simular fallo de abono
76          if (claveDestino.equals(clabeParaFallar)) {
77              throw new RuntimeException("Fallo forzado por variable de entorno para probar la saga");
78          }
79
80          // PUNTO 2: Envolver abono con Circuit Breaker
81          CuentaRemota destino = circuitBreaker.ejecutar(() -> cuentas.abonar(claveDestino, monto), "Abono Destino");
82
```

imagen 12. Cambios importantes transferenciaService

Se modificaron las consultas para conectarlos con el circuit breaker, para que de esta forma cada que se pase de un estado a otro se tenga un registro en los logs. Siendo que se guarda el estado actual y la operación que causa dicho estado.

A su vez, se protege tanto el cargo como el abono, buscando un objeto de una cuenta existente. Devolviendo el saldo actualizado. Además, el circuit breaker captura el objeto y lo demuestra en el hilo principal. Sin embargo, si el microservicio de cuentas está caído, el circuit breaker reconocerá el error en está parte y comprobará si debe entrar en estado abierto de ser necesario.

Extra:

Esta siguiente parte es la que se encarga del rollback, o bien, de la devolución del dinero de la cuenta origen en caso de que la transferencia salga mal, en esta parte el código intenta devolverle el dinero a la cuenta origen hasta 3 veces, a la vez que espera un poco más entre cada intento, empezando en 2 segundos y acabando en 4 segundos para no saturar a los servicios.

En caso de que los 3 intentos fallen, se levanta una alerta para que haya intervención humana y se arregle manualmente el error, avisándole a su vez al usuario que la transferencia falló, pero asegurándole que el dinero no se perdió.

```java
    } catch (Exception e) {
    // 4. ¡EL ABONO FALLÓ! INICIA LA COMPENSACIÓN
    System.err.println("Fallo en el abono, iniciando compensación: " + e.getMessage());

    boolean compensado = false;
    int intentos = 0;
    long esperaMs = 2000; // 2 segundos iniciales

    try {
        // PUNTO 2: Envolver la compensación con Circuit Breaker
        CuentaRemota origenCompensado = circuitBreaker.ejecutar(() -> cuentas.abonar(claveOrigen, monto), "Compensación Origen");

        // Registrar que fue una devolución para que no se vea como un depósito normal
        movimientos.registrar(origenCompensado.clabe(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
            monto, origenCompensado.saldo(), "Devolución por fallo a " + claveDestino);

        transferencia.setEstado("COMPENSADA");
        transferencias.save(transferencia);


    } catch (Exception exCompensacion) {
        // 5. LA COMPENSACIÓN TAMBIÉN FALLÓ (ej. la cuenta se apagó en este momento)
        transferencia.setEstado("COMPENSACION_PENDIENTE");
        transferencias.save(transferencia);
        System.err.println("Fallo CRÍTICO: No se pudo compensar el cargo: " + exCompensacion.getMessage());

        throw new RuntimeException("Fallo grave: Dinero debitado pero no se pudo abonar ni compensar. Queda pendiente de revisión.");
        }

        // Lanzamos una excepción para que el controlador devuelva un 500 o mensaje al usuario
        throw new RuntimeException("La transferencia falló y el dinero fue devuelto a su cuenta. Detalle: " + e.getMessage());
    }

}
```

imagen 13. Función de devolución.

4. La compensación funcionando:

la falla provocada, los saldos que regresan y el estado final en la base.

Para comprobar que la compensación está funcionando utilicé una cuenta específica para que se caiga la transferencia. Esta es:

```yaml
    ▷Run Service
  transferencia:
    <<: *servicio
    build: ./transferencia
    environment:
      <<: *entorno-base
      DB_NAME: transferencia
      CUENTA_URL: http://cuenta
      MOVIMIENTO_URL: http://movimiento
      NOTIFICACION_URL: http://notificacion
      # CLABE para fallos, preparado para que el abono falle y la cuenta origen reciba su dinero de vuelta.
      MEXIBANCO_DEMO_CLABE_FALLO: 000000000000000000
      # Solo para la demo de "lo que se rompio al partir" (ver README de v06a). 0 = sin pausa.
      PAUSA_ENTRE_CARGO_Y_ABONO_MS: ${PAUSA_ENTRE_CARGO_Y_ABONO_MS:-0}

```

imagen 14. Clabe 000000000000000000.

En esta prueba contamos con la transferencia de la CLABE 123456789013245678, que se encargará de transferir 50 pesos de prueba a la cuenta con CLABE 000000000000000000. Para comprobar la compensación adecuadamente veremos el saldo antes y después de la transacción, siendo que la cuenta origen tiene 9850.00 pesos en su cuenta.

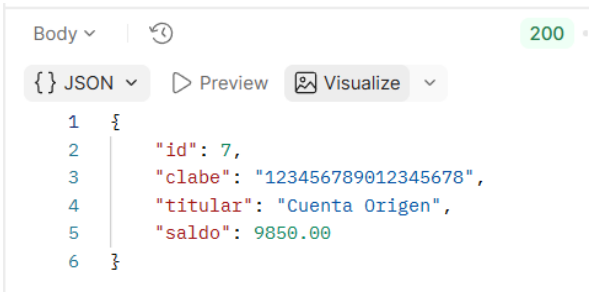


imagen 15. Saldo cuenta origen.

En este caso, al hacer la transferencia, se caerá la misma, debido a que el sistema está condicionado para no aceptar esta clabe. Por lo que Postman muestra el siguiente error 500:

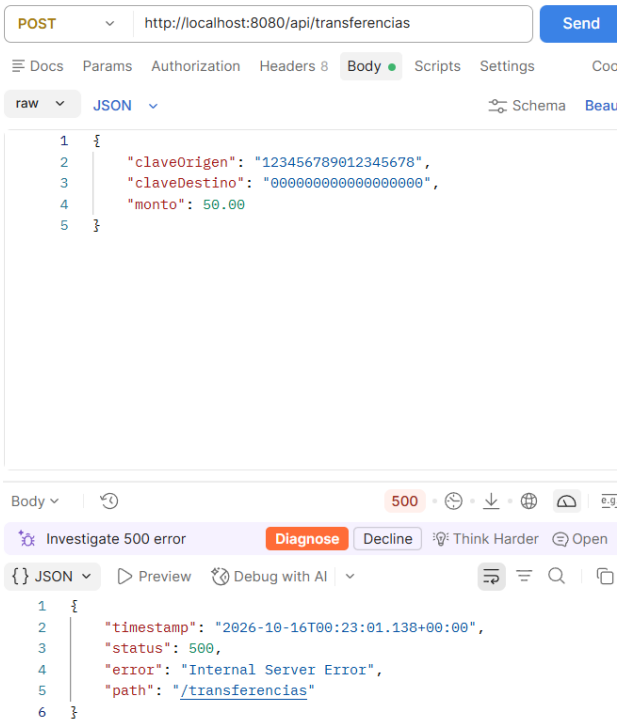


imagen 16. Error de transferencia.

Al obtener este error, se puede saber que la transferencia no fue exitosa, por lo que se detuvo y marcó el error, pero debemos saber donde quedó el dinero. Entrando a la base de datos de postgres para los microservicios encontramos la respuesta:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
psql (17.11)
Type "help" for help.

transferencia=# SELECT * FROM transferencia ORDER BY id DESC LIMIT 1;
 id |   clave_destino    |    clave_origen    |            fecha             | monto |   estado
----+--------------------+--------------------+------------------------------+-------+------------
 14 | 0000000000000000000 | 123456789012345678 | 2026-10-16 00:08:49.626184+00 | 50.00 | COMPENSADA
(1 row)

transferencia=#
```

imagen 17. Transferencia compensada.

De esta forma nos aseguramos de que la transferencia fue compensada y el dinero no se perdió, en la consulta posterior a este movimiento encontramos que el saldo sigue siendo el mismo que antes, confirmando de esta forma que se desarrolló adecuadamente el sistema de compensación en nuestros microservicios:

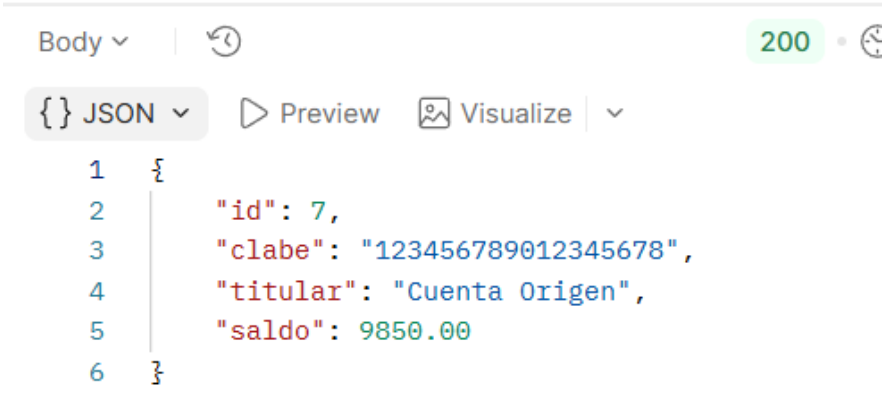


imagen 18. Saldo compensado.

5. Cuando la compensación también falla:

qué estado queda y cómo lo terminarías.

Para esta parte se genera una pausa larga de 20 segundos que nos permite detener correctamente el microservicio cuenta antes de que se concrete la transferencia, esto se realiza a la vez que se reinicia el microservicio transferencia para guardar los cambios:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
[+] up 3/3
 ✔ Container mexi-banco-v06-discovery-1     Healthy
 ✔ Container mexi-banco-v06-db-1            Healthy
 ✔ Container mexi-banco-v06-transferencia-1 Started
```

imagen 19. Reinicio de parametros transferencia.

Ahora, para comprobar lo que ocurre cuando la compensación también falla, guardaremos los saldos antes de la transferencia fallida.

- Origen antes:

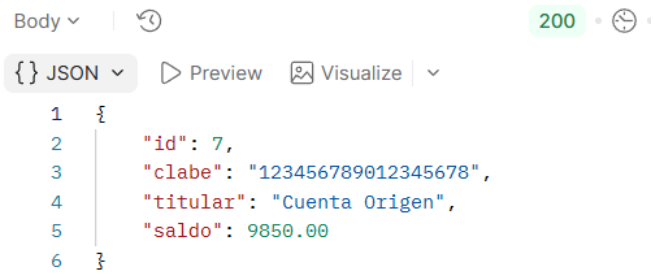


imagen 20. Origen antes del fallo de compensación.

- Destino antes:



imagen 21. Destino antes del fallo de compensación.

Al realizar la petición por postman detuve el microservicio de cuentas para que la compensación se fuerce a fallar también, como podemos ver a continuación, postman nos devuelve un error que indica un fallo de conexión con el microservicio de cuentas:

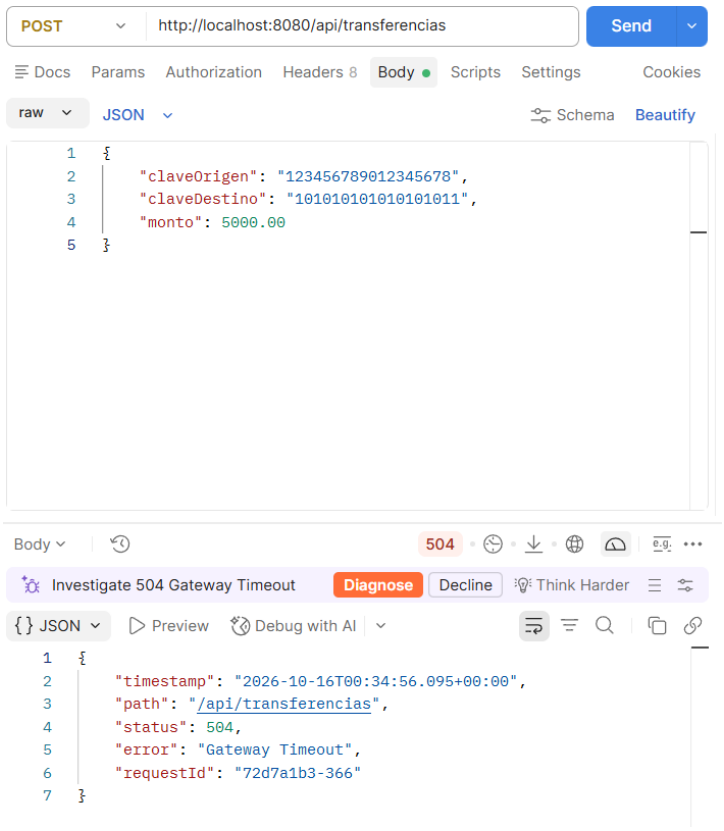


imagen 22. Fallo de comunicación.

Ahora, al revisar la base de datos de postgres del proyecto, encontramos que la compensación fallo y el sistema marcó el estado como una transferencia fallida y una compensación igualmente fallida. Lo que requiere de una intervención humana, siguiendo el esquema que se presentó previamente en el documento:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s08 (03-gateway-discovery-balanceo)
$ docker compose stop cuenta
[+] stop 3/3
 ✔ Container mexi-banco-v06-cuenta-1 Stopped                                    3.2s
 ✔ Container mexi-banco-v06-cuenta-2 Stopped                                    3.3s
psql (17.11)
Type "help" for help.

transferencia=# SELECT * FROM transferencia ORDER BY id DESC LIMIT 1;
 id |   clave_destino    |    clave_origen    |            fecha            |  monto  |         estado
----+--------------------+--------------------+-----------------------------+---------+-----------------------
 16 | 101010101010101011 | 123456789012345678 | 2026-10-16 00:34:41.354376+00 | 5000.00 | COMPENSACION_PENDIENTE
(1 row)

transferencia=#
```

imagen 23. Compensación fallida.

Entonces, si el sistema automático fallo, y se requiere atención humana sobre el movimiento, quiere decir que fue un fallo crítico. Esto lo podemos comprobar con los saldos de cada cuenta involucrada:

- Cuenta Origen:

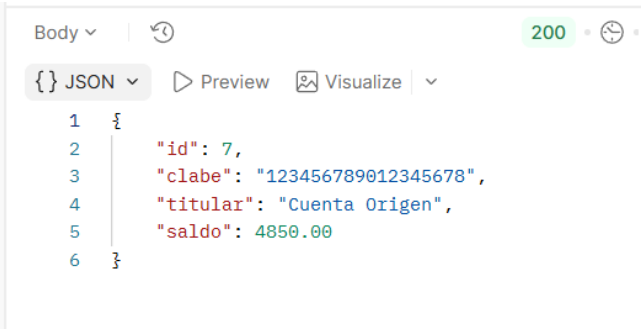


imagen 24. Cuenta Origen en fallo crítico.

- Cuenta Destino:



imagen 25. Cuenta Destino en fallo crítico.

La cuenta origen recibió el cargo por 5000 pesos, sin embargo, la cuenta destino no obtuvo el abono correspondiente, esto es preocupante debido a la perdida del dinero en el vacío, pues, se hizo el cobro correspondiente pero no se efectúo la transferencia. Es por ello que se generó un estado de compensación pendiente, para que un humano pueda resolver lo que el sistema automático no pudo, permitiendo que devuelva el dinero o se complete el movimiento deseado hacia la cuenta destino. A mi parecer esta es la situación adecuada pues, con los logs podemos ver que la transferencia se intentó solucionar automáticamente, pero fallo críticamente. Ante esto la intervención humana a mi consideración es lo adecuado, pues puede resolver la situación y después manipular el estado a completada para terminar la situación.

6. El circuit breaker:

los tres estados, con evidencia.

Ahora, al hacer una transferencia sin ningún problema y todos los microservicios corriendo adecuadamente encontramos este resultado al hacer la petición POST en POSTMAN:

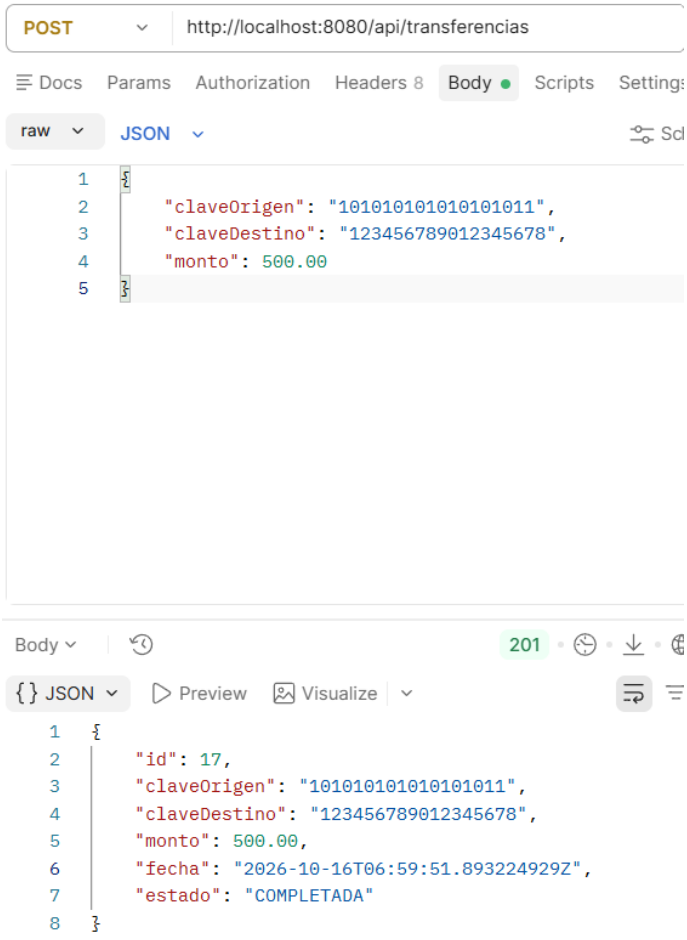


imagen 26. Circuit Breaker Cerrado.

ahora, al hacer de nuevo una transferencia con el microservicio cuenta caído se nos presenta el estado abierto en los logs.

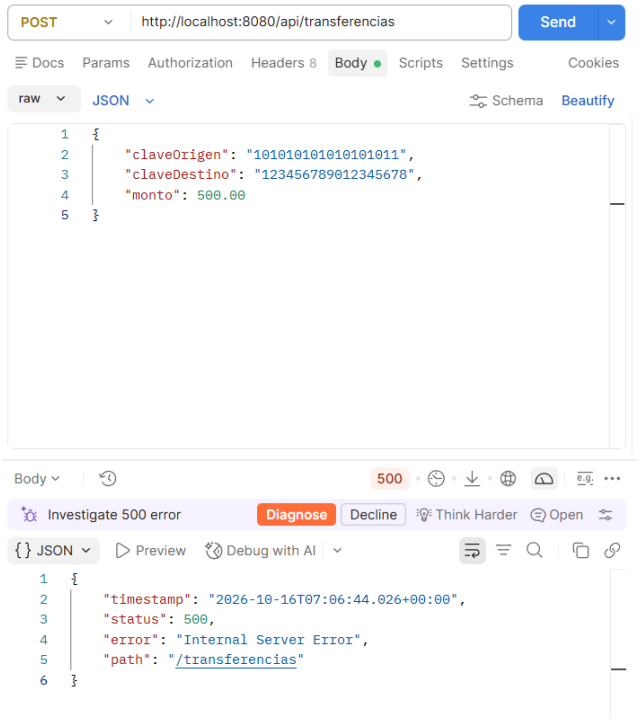


imagen 27. Transferencia fallida.

Se muestra el circuito abierto ya que se cayó el servicio cuenta.

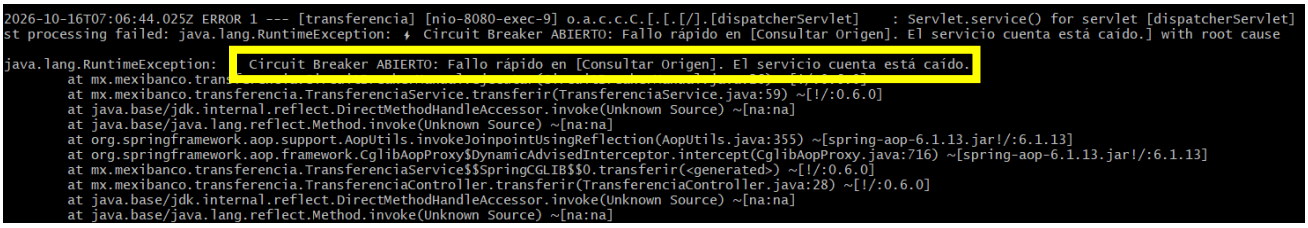


imagen 28. CB, abierto.

Por último, levanté el servicio cuenta para que se recuperé el circuit breaker, pasando del estado abierto al semi-cerrado y por último al cerrado después de comprobar que no hay más errores de comunicación entre microservicios.

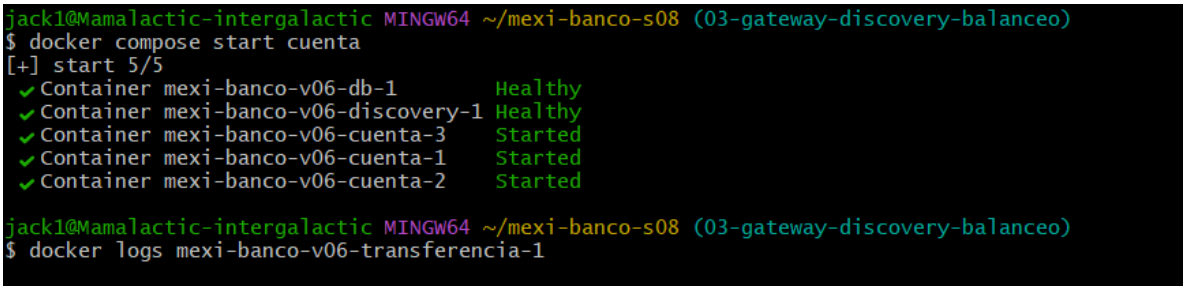


imagen 29. Microservicio cuenta levantado.

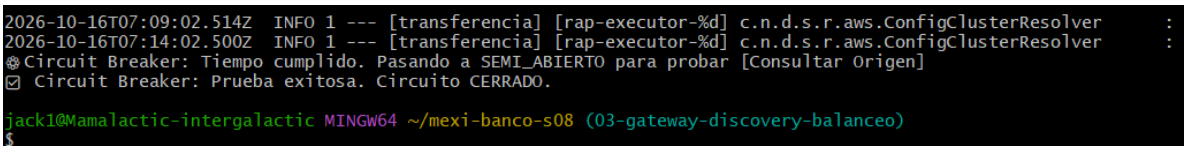


imagen 30. Circuito en cerrado.

Se muestran finalmente todos los estados del circuit breaker, mostrando que el sistema busca llegar al circuito cerrado constantemente cuando detecta un fallo en el mismo sistema.

7. Lo que la saga no te da:

un párrafo sobre el aislamiento (¿qué ve alguien que consulta el saldo de origen entre el cargo y la compensación?).

A diferencia de una transacción tradicional que cumple con las propiedades ACID, el patrón Saga sacrifica el aislamiento (Isolation) para poder operar en sistemas distribuidos. Si un usuario consulta el saldo de su cuenta de origen en el instante exacto entre que se le hizo el cargo y el momento en que ocurre la compensación por fallo (el reverso), verá su saldo reducido. Durante esa ventana de tiempo, el dinero parece estar "en el aire": ya no está en la cuenta origen, pero tampoco ha llegado a la cuenta destino. Esta pérdida de consistencia se acepta en favor de una "consistencia eventual", asumiendo que el sistema corregirá el estado más adelante mediante la compensación.